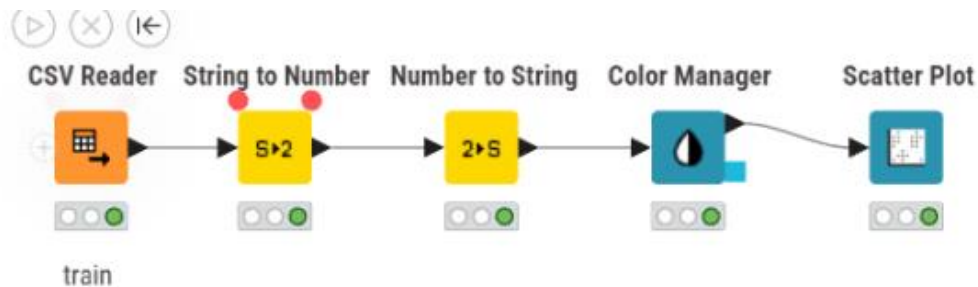


1. Visualizar as 2 luas



Objetivo:

Visualizar graficamente o dataset moon_or_not_moon usando o KNIME. O objetivo é representar os pontos de dados no plano XX/YY, com cores distintas para cada classe (0 e 1), formando a forma característica de duas luas.

Fluxo KNIME utilizado:

CSV Reader (train) → String to Number → Number to String → Color Manager → Scatter Plot

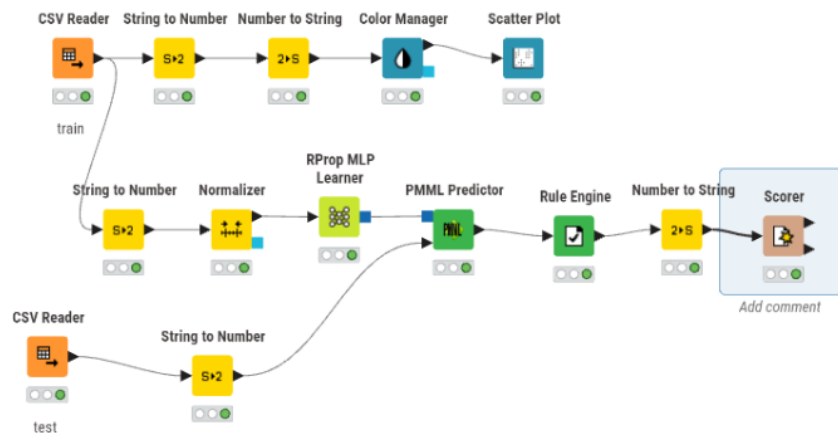
Explicacao de cada nodo:

- 1. CSV Reader:** Carrega o ficheiro train.csv com as colunas XX (coordenada horizontal), YY (coordenada vertical) e class (classificador 0 ou 1).
- 2. String to Number:** Converte as colunas XX e YY de texto para numeros reais (Double), necessario para usar no Scatter Plot.
- 3. Number to String:** Converte a coluna class de numero para String. Isto é necessario para que o Color Manager reconheca a classe como nominal (categorica) e não como continua, permitindo atribuir cores distintas a cada classe.
- 4. Color Manager:** Atribui cores aos pontos com base na coluna class. Configuracao: Color by = class, modo Nominal, classe 0 = verde, classe 1 = vermelho.
- 5. Scatter Plot:** Representa graficamente os dados. Configuracao: Horizontal dimension = XX, Vertical dimension = YY, Color dimension = class. O resultado mostra as duas luas em cores distintas.

Nota: O problema das duas luas é não-linearmente separável - nenhuma linha reta consegue separar as duas classes, o que torna este dataset ideal para testar redes neurais.

2. Adicionar uma Rede Neuronal para classificar

Scatter Plot



Rows: 2 | Columns: 2

Table  Statistics 

☐	#	RowID	0 Number (Integer)	1 Number (Integer)
☐	1	0	384	113
☐	2	1	32	471

☐	#	RowID	TruePosit... Number (Integer)	FalsePosi... Number (Integer)	TrueNeg... Number (Integer)	FalseNeg... Number (Integer)	Recall Number (Double)	Precision Number (Double)	Sensitivity Number (Double)	Specificity Number (Double)	F-measure Number (Double)
☐	1	0	384	32	471	113	0.773	0.923	0.773	0.936	0.841
☐	2	1	471	113	384	32	0.936	0.807	0.936	0.773	0.867

Objetivo:

Treinar uma rede neuronal artificial (RNA) para classificar os pontos do dataset nas duas classes (lua 0 e lua 1), e depois avaliar o desempenho do modelo usando o dataset de teste.

Fluxo KNIME completo:

TREINO:

CSV Reader (train) -> String to Number (XX e YY) -> Normalizer -> RProp MLP Learner

TESTE E AVALIACAO:

CSV Reader (test) -> String to Number -> Normalizer (Apply) -> PMML Predictor -> Rule Engine -> Scorer

Explicacao de cada nodo:

1. String to Number (so XX e YY): A coluna class deve ser mantida como String para que o RProp MLP Learner a reconheca como classificacao e nao regressao. So XX e YY sao convertidos para Double.

2. Normalizer: Normaliza XX e YY para o intervalo [0,1] usando Min-Max Normalization. As redes neuronais funcionam melhor com dados normalizados porque evita que valores maiores dominem o treino. A segunda saida do Normalizer e ligada ao Normalizer Apply do fluxo de teste.

3. RProp MLP Learner: Treina a rede neuronal. Parametros configurados: Max iterations = 100, Hidden layers = 1, Neurons per layer = 10, Class column = class. O algoritmo RProp (Resilient Backpropagation) ajusta os pesos da rede iterativamente para minimizar o erro de classificacao.

4. Normalizer Apply: Aplica a mesma normalizacao do treino aos dados de teste. E essencial usar os mesmos parametros (min/max do treino) para que os dados de teste estejam na mesma escala.

5. PMML Predictor: Aplica o modelo treinado aos dados de teste. Recebe duas entradas: o modelo PMML (saida do Learner) e os dados normalizados de teste. Gera a coluna Prediction (class) com valores continuos entre 0 e 1.

6. Rule Engine: Converte os valores continuos da previsao (ex: 0.87) em classes discretas. Regra: se Prediction (class) maior ou igual a 0.5 entao 1.0, caso contrario 0.0. Cria a coluna prediction com o mesmo tipo que class.

7. Scorer: Avalia o desempenho do modelo comparando a class real com a prediction. Gera a matriz de confusao e metricas: Accuracy, Precision, Recall, F-measure.

Resultados obtidos (RProp MLP - 1 camada, 10 neuronios, 100 iteracoes):

Matriz de confusao: TN = 384, FP = 113, FN = 32, TP = 471 (total = 1000 casos de teste)

Accuracy = 85.5% | Precision = 80.7% | Recall = 93.6%

Nota: Para melhorar os resultados, experimenta aumentar o numero de camadas (ex: 2), neuronios (ex: 20-50) e iteracoes (ex: 200-500). Para a tarefa 4 do guia, compara com Decision Tree, Logistic Regression e outros algoritmos usando o mesmo fluxo de avaliacao.

3. Resumo Teorico - RNA Treino Linear (aula)

O que e uma RNA (Rede Neuronal Artificial)?

Uma RNA e um modelo computacional inspirado no cerebro humano, composto por neuronios artificiais interligados. Cada neuronio recebe entradas, aplica uma funcao de ativacao e produz uma saida. A rede aprende ajustando os pesos (P) das ligacoes entre neuronios.

Arquitetura da rede dos slides (exemplo):

A rede apresentada na aula tem a seguinte estrutura:

- **Camada de entrada:** 2 neuronios (A e B), que recebem as entradas INA=0 e INB=1
- **Camada oculta:** 3 neuronios (C, D e E), totalmente ligados aos neuronios de entrada
- **Camada de saida:** 1 neuronio (F) que produz OUTF. O valor esperado (desejado) e 1.

Propagacao para a frente (Forward Pass):

Cada neuronio calcula a sua ativacao usando: $fA(P,E) = \text{Soma}(P \times E)$, onde P sao os pesos e E as entradas. A funcao de transferencia linear e $fT(A) = A$ (a saida e igual a ativacao, sem transformacao).

Exemplo do slide (pesos iniciais, INA=0, INB=1): C recebe A(peso 1,1) + B(peso -1,4) = -1,4 | D recebe A(peso 3,6) + B(peso -4,1) = -4,1 | E recebe A(peso 2,1) + B(peso 2,5) = 2,5 | F recebe C(peso 1,2) + D(peso 1,6) + E(peso 4,3) = 2,5

Calculo do Erro e Retropropagacao (Backpropagation):

Passo 1 - Calcular o erro na saida: $\text{epsilon} = \text{OUT_Desejado} - \text{OUT_Calculado} = 1 - 2,5 = -1,5$

Passo 2 - Propagar o erro para tras: $\text{epsilon_back} = \text{epsilon} \times P$ (multiplicar o erro pelo peso de cada ligacao). Exemplo: erro em C = $-1,5 \times 1,2 = -1,8$ | erro em D = $-1,5 \times 1,6 = -2,4$ | erro em E = $-1,5 \times 4,3 = -6,5$

Passo 3 - Atualizar os pesos: $P_{\text{novo}} = P_{\text{antigo}} + \text{epsilon} \times fT$. Exemplo: peso A->C: $1,1 + (-1,8) \times (-1,4) = 1,1 + 2,52 = 3,6$ (novo peso)

Iteracoes sucessivas:

Apos a atualizacao dos pesos, a rede faz uma nova propagacao forward com os pesos atualizados. O processo repete-se (iteracoes) ate o erro convergir para zero ou atingir um maximo de iteracoes. No slide 14 e 15, apos a segunda iteracao, os pesos mudam

drasticamente (ex: D passa de -4,1 para 13,4) e o output de F passa para -23,7. O erro aumentou antes de diminuir - e normal no inicio do treino.

4. Conclusao

O problema das duas luas e as RNAs:

O dataset moon_or_not_moon e um problema de classificacao binaria nao-linearmente separavel: as duas classes (0 e 1) formam duas luas interligadas que nenhuma linha reta consegue separar. Isto torna-o um excelente caso de estudo para redes neuronais artificiais, que conseguem aprender fronteiras de decisao curvas e complexas.

Ligacao entre a teoria (slides) e a pratica (KNIME):

- **Forward Pass (slides):** o KNIME executa automaticamente a propagacao forward no RProp MLP Learner em cada iteracao
- **Calculo do erro (slides: $\epsilon = \text{OUT}_D - \text{OUT}_C$):** o KNIME calcula o erro entre o valor previsto e o real em cada iteracao
- **Atualizacao dos pesos (slides: $P_{i+1} = P_i + \epsilon \times \text{ft}$):** o algoritmo RProp faz esta atualizacao automaticamente, com a diferenca de usar um metodo adaptativo que ajusta o tamanho do passo
- **Max iterations=100 (KNIME):** equivale a repetir o ciclo forward+backpropagation 100 vezes, tal como os slides mostram iteracao a iteracao

Resultados e comparacao de tecnicas:

A RNA (RProp MLP) atingiu 85,5% de accuracy com apenas 1 camada oculta e 10 neuronios. Este resultado e esperado ser superior ao de metodos lineares como a Regressao Logistica (que nao consegue separar classes nao-lineares) e competitivo com a Decision Tree. Para o problema das duas luas, metodos com fronteiras nao-lineares (RNA, k-NN, SVM com kernel) devem superar os lineares.

Dificuldades encontradas e solucoes:

1. **Color Manager - opcao Nominal bloqueada:** solucao - adicionar Number to String antes do Color Manager para converter class para String
2. **Scatter Plot sem cores:** solucao - configurar Color dimension = class nas opcoes do Scatter Plot
3. **RProp MLP fazia regressao em vez de classificacao:** solucao - nao converter a coluna class para numero, mante-la como String para o Learner a reconhecer como classificacao
4. **Scorer com tipos incompativeis:** solucao - usar Rule Engine para converter a previsao continua em classe discreta, garantindo o mesmo tipo de dados em ambas as colunas comparadas